

Technical Challenge: Omnichannel Broadcast Engine

Overview

At **3Dolphins**, our core product manages the end-to-end business lifecycle. A critical part of this is our **Omnichannel Messaging Engine**, which handles high-volume communication across platforms like WhatsApp and Facebook.

The Goal: Build a "Priority Broadcast" prototype that demonstrates your ability to handle complex UI state, asynchronous processing, and third-party integrations using **Java** and **PrimeFaces**.

The Scenario

You are tasked with building a module that allows a user to select a group of customers and send a broadcast message. Because external APIs (like WhatsApp) can be slow or fail, the system must handle these calls **asynchronously** to ensure the user interface remains responsive.

Technical Requirements

1. The Frontend (PrimeFaces)

Create a single page (`broadcast.xhtml`) that includes:

- **Customer Selection:** A `p:dataTable` containing a list of 10 mock customers (Name, Phone Number, Status).
- **The Action:** A `p:commandButton` labeled "Start Priority Broadcast."
- **The Feedback:** While the broadcast is running, the UI must show a **real-time progress indicator** (e.g., a `p:progressBar` or status updates in the table).
- **Constraint:** The broadcast must be triggered via **AJAX**. The page should not perform a full refresh during the process.

2. The Backend (Java)

- **Mock Integration Service:** Create a service that simulates an external API call.
 - Each call should have a random delay of **1–2 seconds**.
 - Simulate a **10% failure rate** (where the message status becomes "Failed").
- **Asynchronous Execution:** The broadcast logic must run on a background thread. The UI thread must not be blocked while the "API calls" are in progress.
- **State Management:** Choose the appropriate JSF Scope (e.g., `@ViewScoped`, `@SessionScoped`) to manage the state of the broadcast.

3. Challenge Question

Please include a brief text file answering the following:

1. **Architecture:** Why did you choose your specific threading/asynchronicity model (e.g., `CompletableFuture`, `@Asynchronous`, etc.)?
 2. **Scalability:** If this broadcast had 50,000 recipients instead of 10, what changes would you make to your current implementation to ensure system stability?
 3. **Resilience:** How would you handle a "Rate Limit" error from the external API to ensure no messages are lost?
-

Evaluation Criteria

- **Lifecycle Mastery:** Correct use of PrimeFaces `process` and `update` attributes.
- **Thread Safety:** Ensuring that background updates to the UI state are handled safely.
- **Code Quality:** Clean separation of concerns (UI vs. Business Logic vs. Integration).
- **Aptitude:** Even if you are new to PrimeFaces, we are looking for how quickly you can leverage its documentation to build a functional UI.

Submission Instructions

- Please provide the source code (Maven or Gradle project) via a Git repository link or a ZIP file.
 - Include instructions on how to run the application (e.g., which application server or Docker setup).
 - **Time Limit:** We expect this should take approximately **3–5 hours** of focused work. Please submit within **48 hours** of receiving this invite.
-